

Die Java-API kennenlernen – Teil 2

Erforschen der API | Textbasiertes Interface

Programmierung 1

PIB-PR1 / DFIW-PRG1

Software Engineering und Software Quality Assurance { SESQA }

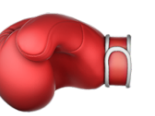
Prof. Dr. Martin Burger { 4. Dezember 2025 }

inno



Plan der Präsenzveranstaltungen

VW	KW	 Inhalte in der „Vorlesung“	Mo	Di	Mi	Do
1	42	Willkommen und erste Orientierung, Lebenslanges Lernen				
2	43	Einstieg in Java (Kap. 1)				
3	44	Teamarbeit				
4	45	Klassen und Objekte (Kap. 2)				
5	46	Elementare Datentypen und Referenzen (Kap. 3)				
6	47	Methoden benutzen Instanzvariablen (Kap. 4)				
7	48	Ein Programm schreiben (Kap. 5)				
8	49	Die Java-API kennenlernen (Kap. 6)				
9	50	Vererbung und Polymorphie (Kap. 7)				
10	51	Interfaces und abstrakte Klassen (Kap. 8)				
	52	Vorlesungsfreie Zeit (Jahreswechsel)				
	01	Vorlesungsfreie Zeit (Jahreswechsel)				
11	02	Konstruktoren und Garbage Collection (Kap. 9)				
12	03	Zahlen und Statisches (Kap. 10)				
13	04	Exception-Handling (Kap. 13)				
14	05	Serialisierung und Datei-I/O (Kap. 16)				
15	06	Fragen und Antworten (Puffer)				

„Kein Plan 
überlebt den
ersten
Kontakt  mit
der Realität .

Frei nach Helmuth Karl
Bernhard von Moltke

Die Java-API kennenlernen

 Plan für diese Woche

 ArrayList

 Boolesche Ausdrücke

 Pair Programming

 Packages

 Erforschen der API

 Textbasiertes Interface

 Spaß

📖 Die Java-API kennenlernen

🔗 Verbindungen herstellen

👤 Tauschen Sie sich mit einer Nachbar:in aus!

🌐 Wie finden Sie heraus, welche Features oder Klassen Java zur Verfügung stellt, wie Sie diese nutzen können und was diese konkret können?

💻 Was von Ihrem bisherigen Wissen über Java können Sie einsetzen, um mit der Benutzer:in Ihres Programms zu interagieren?

🙋 Welche Fragen haben Sie zu diesem Kapitel?

6 Die Java-API kennenlernen

Die Java-Bibliothek verwenden



Java wird mit Hunderten vorgefertigter Klassen ausgeliefert. Sie müssen das Rad nicht neu erfinden, wenn Sie wissen, wie Sie das Benötigte in der Java-Bibliothek, der sogenannten **Java-API**, finden können. *Sie haben Besseres zu tun.* Wenn Sie Code schreiben, reicht es völlig, wenn Sie sich *nur auf die Teile* konzentrieren, die bei Ihrer Anwendung besonders sind. Kennen Sie diese Programmierer, die jeden Tag um Punkt 5 nach Hause gehen und am nächsten Tag nicht vor 10 wieder auftauchen? **Sie benutzen die Java-API.** Und ungefähr acht Seiten weiter werden auch Sie das tun. Die Java-Kernbibliothek (Core Library) ist ein gigantischer Haufen Klassen, die nur darauf warten, dass Sie sie als Bausteine für Ihre eigenen Programme aus größtenteils vorgefertigtem Code verwenden. Die Java-Fertiggerichte, die wir in diesem Buch einsetzen, bestehen aus Code, den Sie nicht von Grund auf neu schreiben müssen. Eingeben müssen Sie ihn trotzdem. Die Java-API ist voller Code, den Sie nicht einmal *eingeben* müssen. Sie müssen nur lernen, wie man ihn benutzt.

Hier fängt ein neues Kapitel an 125



Erforschen der API

Die Java-Bibliothek

Erforschen der API

 Java kommt mit Hunderten von vorgefertigten Klassen – der Java-Bibliothek.

 Sie müssen das Rad nicht neu erfinden. Sie benötigen nur das **Wissen, wie** und **wo** Sie das finden, was Sie aus der Java-Bibliothek brauchen.

 **Java-API** (Application Programming Interface): Eine Sammlung von standardisierten, ausgereiften und dokumentierten Klassen, Schnittstellen und Methoden, mit denen Sie schnell Java-Anwendungen erstellen können.

 Wenn Sie Code schreiben, können Sie sich auf das konzentrieren, was für Ihre Anwendung **einzigartig** ist.



„Ja, ja, ich weiß! Wie zum Beispiel ArrayList! Ich spare mir die Implementierung einer eigenen Liste, die nach Belieben schrumpfen oder wachsen kann.“

Gerit – aufgeweckte Student:in im 1. Semester



„Okay – und welche Features und Klassen stellt die Java-API noch zur Verfügung?“

Noel – neugierige Student:in im 1. Semester



O'REILLY®

8th Edition
Covers Java 17

Java in a Nutshell

A Desktop Quick Reference



Benjamin J Evans,
Jason Clark &
David Flanagan

Wie kann ich
das Feature
verwenden?
Was kann
die Klasse?

Part II. Working with the Java Platform

7. Programming and Documentation Conventions.....	267
Naming and Capitalization Conventions	267
Practical Naming	269
Java Documentation Comments	271
Doclets	279
Conventions for Portable Programs	280
Summary	282
8. Working with Java Collections.....	283
Introduction to Collections API	283
Java Streams and Lambda Expressions	306
Summary	317
9. Handling Common Data Formats.....	319
Text	319
Numbers and Math	329
Date and Time	334
Summary	341
10. File Handling and I/O.....	343

OVERVIEW

MODULE

PACKAGE

CLASS

USE

TREE

PREVIEW

NEW

DEPRECATED

INDEX

HELP

SEARCH

Java® Platform, Standard Edition & Java Development Kit Version 21 API Specification

This document is divided into two sections:

Java SE

The Java Platform, Standard Edition (Java SE) APIs define the core Java platform for general-purpose computing. These APIs are in modules whose names start with `java`.

JDK

The Java Development Kit (JDK) APIs are specific to the JDK and will not necessarily be available in all implementations of the Java SE Platform. These APIs are in modules whose names start with `j`.

All Modules	Java SE	JDK	Other Modules
Module	Description		
<code>java.base</code>	Defines the foundational APIs of the Java SE Platform.		
<code>java.compiler</code>	Defines the Language Model, Annotation Processing, and Java Compiler APIs.		
<code>java.datatransfer</code>	Defines the API for transferring data between and within applications.		
<code>java.desktop</code>	Defines the AWT and Swing user interface toolkits, plus APIs for access to native operating system features, printing, and JavaBeans.		
<code>java.instrument</code>	Defines services that allow agents to instrument programs running on the JVM.		

Help Center

Java Core Libraries Developer Guide

Search

Java / Java SE / 21

Core Libraries

Expand

Title and Copyright Information

Preface

1 Java Core Libraries

2 Serialization Filtering

3 Enhanced Deprecation

4 XML Catalog API

5 Java Collections Framework

6 Process API

7 Preferences API

8 Java Logging Overview

9 Java NIO

10 Java Networking

11 Pseudorandom Number Generators

12 Foreign Function and Memory API

1 Java Core Libraries

The core libraries consist of classes which are used by many portions of the JDK. They include functionality which is close to the VM and is not explicitly included in other areas, such as security. Here you will find current information that will help you use some of the core libraries.

Topics in this Guide

• [Serialization Filtering](#)

• [Enhanced Deprecation](#)

• [XML Catalog API](#)

• [Java Collections Framework](#)

• [Process API](#)

• [Preferences API](#)

• [Java Logging Overview](#)

• [Java NIO](#)

• [Java Networking](#)

• [Pseudorandom Number Generators](#)

• [Foreign Function and Memory API](#)

1 Java Core Libraries

⌕	Title and Copyright Information
📖	Preface
☰	1 Java Core Libraries
▶	2 Serialization Filtering
▶	3 Enhanced Deprecation
▶	4 XML Catalog API
▼	5 Java Collections Framework
	Creating Unmodifiable Lists, Sets, and Maps
	Creating Sequenced Collections, Sets, and Maps
▶	6 Process API
▶	7 Preferences API
	8 Java Logging Overview
▶	9 Java NIO
	10 Java Networking
▶	11 Pseudorandom Number Generators
▶	12 Foreign Function and Memory API
	13 Scoped Values

5 Java Collections Framework

The Java platform includes a collections framework that provides develop for representing and manipulating collections, enabling them to be manipulated details of their representation. A collection is an object that represents a classic `ArrayList` class).

The [Java Collections Framework](#) enables interoperability among unrelated designing and learning new APIs, and fosters software reuse. The framework dozen collection interfaces, and includes implementations of these interfaces manipulate them.

Overview

The Java Collections Framework consists of:

- **Collection interfaces:** Represent different types of collections, such as `Collection`. These interfaces form the basis of the framework.
- **General-purpose implementations:** Primary implementations of the collection interfaces.
- **Legacy implementations:** The collection classes from earlier releases were retrofitted to implement the collection interfaces.
- **Special-purpose implementations:** Implementations designed for specific use cases. These implementations display nonstandard performance characteristics or behavior.
- **Concurrent implementations:** Implementations designed for high concurrency.
- **Wrapper implementations:** Add functionality, such as synchronization, to existing collections.
- **Convenience implementations:** High-performance "mini-implementations" of the collection interfaces.

OVERVIEWMODULEPACKAGECLASSUSETREEPREVIEWNEWDEPRECATEDINDEXHELP

PACKAGE: DESCRIPTION | RELATED PACKAGES | CLASSES AND INTERFACES

SEARCH

Java Collections Framework

For an overview, API outline, and design rationale, please see:

- [Collections Framework Documentation](#)

For a tutorial and programming guide with examples of use of the collections framework, please see:

- [Collections Framework Tutorial](#)

Since:
1.0

Related Packages

Module	Package	Description
java.base	java.util.concurrent	Utility classes commonly useful in concurrent programming.
java.base	java.util.function	<i>Functional interfaces</i> provide target types for lambda expressions and method references.
java.base	java.util.jar	Provides classes for reading and writing the JAR (Java ARchive) file format. ZIP file format with an optional manifest file.
java.logging	java.util.logging	Provides the classes and interfaces of the Java 2 platform's core logging framework.
java.prefs	java.util.prefs	This package allows applications to store and retrieve user and system preferences.
java.base	java.util.random	This package contains classes and interfaces that support a generic API for random number generation.
java.base	java.util.regex	Classes for matching character sequences against patterns specified by regular expressions.
java.base	java.util.spi	Service provider classes for the classes in the java.util package.
java.base	java.util.stream	Classes to support functional-style operations on streams of elements, such as collections.

	operations.
AbstractSequentialList <E>	This class provides a skeletal implementation of the List interface to minimize the effort required to implement this interface backed by a "sequential access" data store (such as a linked list).
AbstractSet <E>	This class provides a skeletal implementation of the Set interface to minimize the effort required to implement this interface.
ArrayDeque <E>	Resizable-array implementation of the Deque interface.
ArrayList <E>	Resizable-array implementation of the List interface.
Arrays	This class contains various methods for manipulating arrays (such as sorting and searching).
Base64	This class consists exclusively of static methods for obtaining encoders and decoders for the Base64 encoding scheme.
Base64.Decoder	This class implements a decoder for decoding byte data using the Base64 encoding scheme as specified in RFC 4648 and RFC 2045.
Base64.Encoder	This class implements an encoder for encoding byte data using the Base64 encoding scheme as specified in RFC 4648 and RFC

Module java.base
Package java.util

Class ArrayList<E>

java.lang.Object
 java.util.AbstractCollection<E>
 java.util.AbstractList<E>
 java.util.ArrayList<E>

Type Parameters:
E - the type of elements in this list

All Implemented Interfaces:
Serializable, Cloneable, Iterable<E>, Collection<E>, List<E>, RandomAccess, SequencedCollection<E>

Direct Known Subclasses:
AttributeList, RoleList, RoleUnresolvedList

```
public class ArrayList<E>  
  extends AbstractList<E>  
  implements List<E>, RandomAccess, Cloneable, Serializable
```

Resizable-array implementation of the List interface. Implements all optional list operations, and permits all elements, including null. In addition to implementing the List interface, this class provides methods to manipulate the size of the array that is used internally to store the list. (This class is roughly equivalent to Vector, except that it is unsynchronized.)

E	set(int index, E element)	Replaces the element at the specified position in this list with the specified element.
int	size()	Returns the number of elements in this list.
Splitterator<E>	spliterator()	Creates a <i>late-binding</i> and <i>fail-fast</i> Spliterator over the elements in this list.
List<E>	subList(int fromIndex, int toIndex)	Returns a view of the portion of this list between the specified fromIndex, inclusive, and toIndex, exclusive.
Object[]	toArray()	Returns an array containing all of the elements in this list in proper sequence (from first to last element).
<T> T[]	toArray(T[] a)	Returns an array containing all of the elements in this list in proper sequence (from first to last element); the runtime type of the returned array is that of the specified array.
void	trimToSize()	Trims the capacity of this ArrayList instance to be the list's current size.

toArray

```
public Object[] toArray()
```

Returns an array containing all of the elements in this list in proper sequence (from first to last element).

The returned array will be "safe" in that no references to it are maintained by this list. (In other words, this method must allocate a new array). The caller is thus free to modify the returned array.

This method acts as bridge between array-based and collection-based APIs.

Specified by:

toArray in interface `Collection<E>`

Specified by:

toArray in interface `List<E>`

Overrides:

toArray in class `AbstractCollection<E>`

Returns:

an array containing all of the elements in this list in proper sequence

See Also:

`Arrays.asList(Object[])`

🧑 „Und zwar so, dass
ich es selbst verstehe!“

„Stellen Sie sich ständig die Frage: Wie
erreiche ich die Lösung – mit möglichst wenig
und möglichst nachvollziehbarem Code?“

Maxi – professionelle Entwickler:in, die andere zu Spitzenleistungen führt



! ? Quiz

Erforschen der API

-  Ich mache einige Aussagen – sind diese Aussagen richtig oder falsch?
-  Bei **richtigen** Aussagen **stehen** Sie **kurz auf**!
-  Bei **falschen** Aussagen **bleiben** Sie **sitzen**!



Java wird mit Hunderten von vorgefertigten Klassen ausgeliefert, die als Java Library (dt. Bibliothek) oder Java-API (Java Application Programming Interface) bezeichnet werden.

Richtig!

Diese Java-API ist eine Sammlung von standardisierten, gründlich getesteten und dokumentierten Klassen, Schnittstellen und Methoden, mit denen Sie schnell Java-Anwendungen erstellen können.

Richtig!

Alles, was Sie brauchen, ist das Wissen, wie und wo Sie das finden, was Sie aus der Java-Bibliothek benötigen.

Richtig!

Der einzig richtige Weg, mit der Recherche zu beginnen, ist das Buch Java in a Nutshell zu lesen.

Es gibt viele Möglichkeiten.

Die Java-API ist in der „HTML-API-Dokumentation“ beschrieben. Dies ist die offizielle Referenzdokumentation.

Richtig!

Eine wichtige Fähigkeit als professionelle Programmierer:in ist es, dass Sie das Rad nicht jedes Mal neu erfinden. Sie greifen auf bestehende, ausgereifte Funktionalitäten zurück, die Sie mit großem Sachverstand einsetzen.

Richtig!

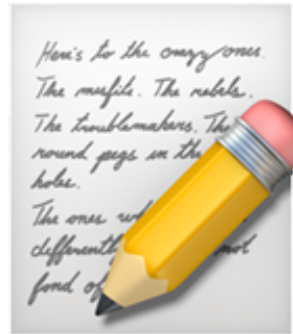


Welcher **eine** Gedanke zu



„Erforschen der API“ war

für Sie am wichtigsten?



Schreiben Sie ihn auf!



Durchatmen und



Sortieren

Textbasiertes Interface

„Implementieren Sie eine interaktive
Benutzeroberfläche für den Umrechner
zwischen Celsius und Fahrenheit!“

Ellis – begeisterte Kund:in, die den Umrechner im OneCompiler entdeckt hat



Anforderungen

Textbasiertes Interface

-  „Ich möchte über die Konsole umrechnen können.“
-  „Ich möchte nach jeder Umrechnung entscheiden können, ob ich noch einmal umrechnen oder den Umrechner beenden möchte.“
-  „Wenn ich erst einmal interaktiv Temperaturen umrechnen kann, werde ich mir sicher noch weitere Features ausdenken...“



„Über Konsole umrechnen, nach jeder Umrechnung entscheiden und weitere Features – Wie würden Sie das umsetzen?“

Maxi – professionelle Entwickler:in, die andere zu Spitzenleistungen führt





Umsetzung



Textbasiertes Interface



„Sie möchte über die Konsole umrechnen können.“



„Sie möchte nach jeder Umrechnung entscheiden können, ob Sie noch einmal umrechnen oder den Umrechner beenden möchte.“



„Wenn Sie erst einmal interaktiv Temperaturen umrechnen kann, wird Sie sich sicher noch weitere Features ausdenken...“

Command Line Interface (CLI)

Menu mit Auswahloptionen

Prinzip „Easier to Change“:
Gutes Softwaredesign ist leichter zu ändern als schlechtes Softwaredesign.

Easier to Change: Niedrige Kopplung

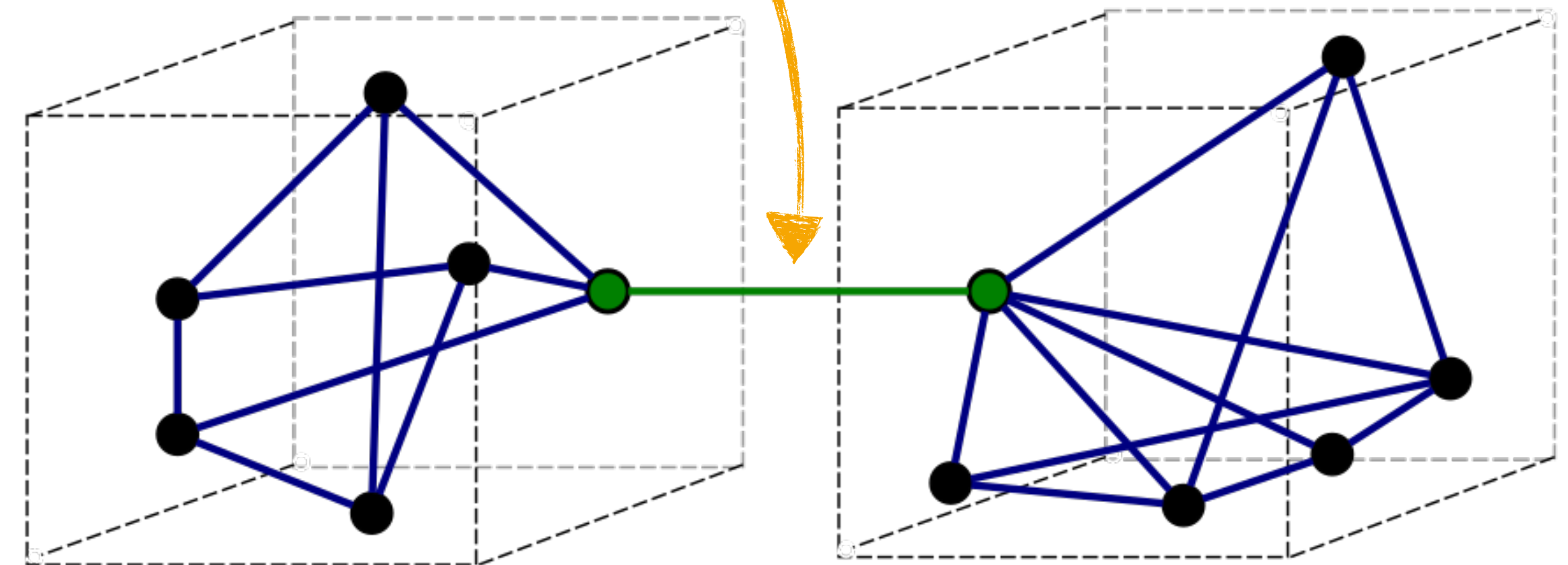
Textbasiertes Interface

💡 **Kopplung** bezieht sich auf das Ausmaß, in dem einzelne Module (Methoden, Klassen, Packages, ...) einer Software miteinander verbunden sind.

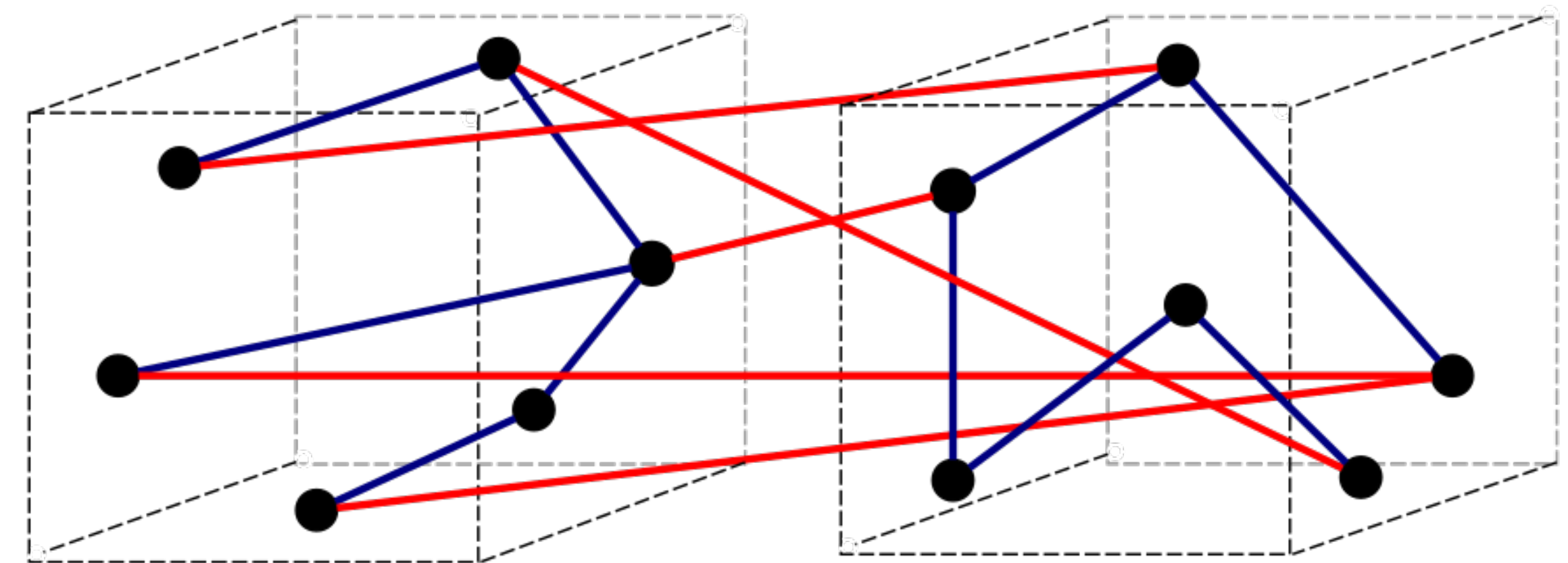
🔗 **Niedrige Kopplung:** Module in einer Software sind möglichst unabhängig voneinander.

🔨 Änderungen in einem Modul sollen möglichst geringe Auswirkungen auf andere Module haben.

💻 Niedrige Kopplung fördert die Flexibilität und Wartbarkeit in der Softwareentwicklung.



a) Good (loose coupling, high cohesion)

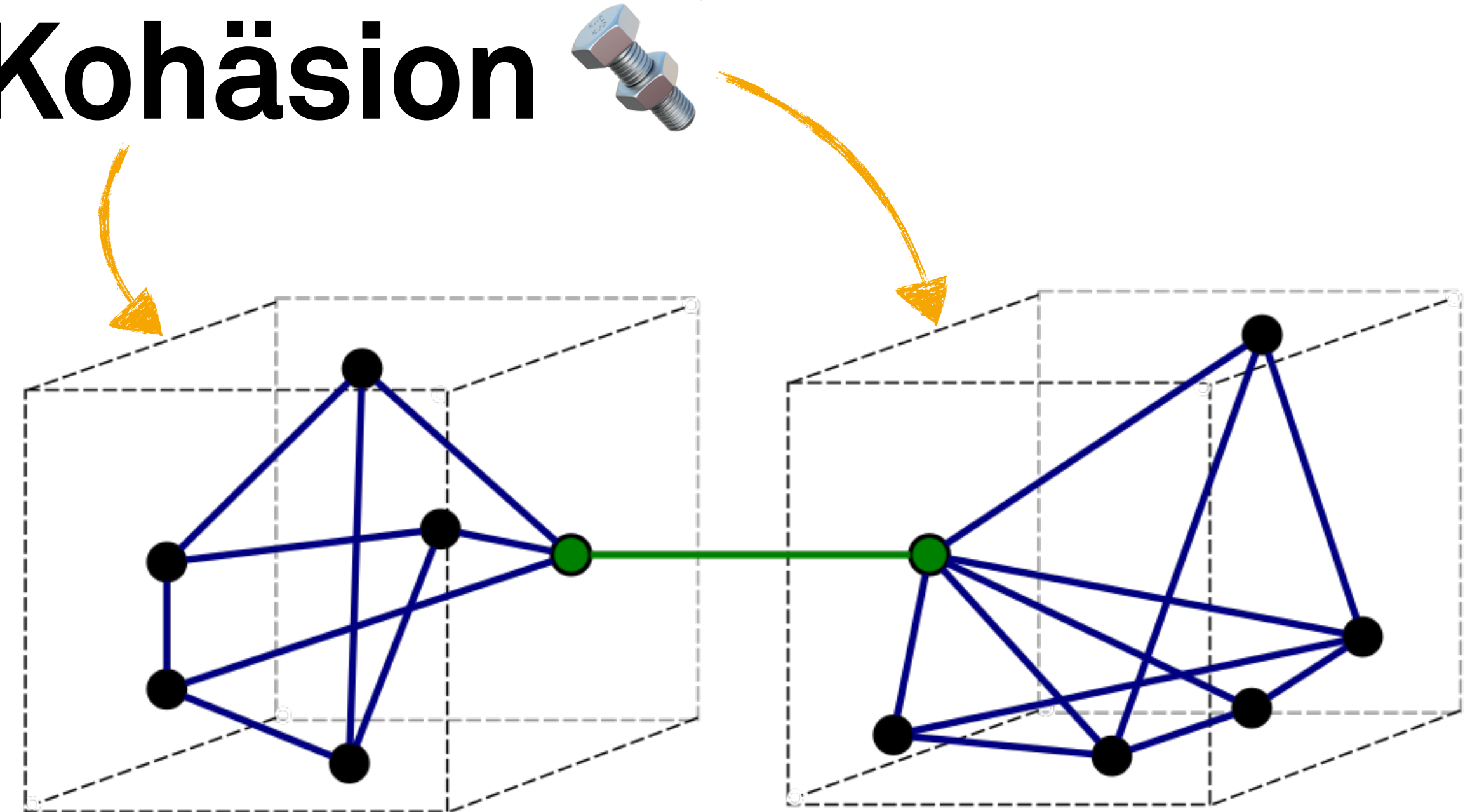


b) Bad (high coupling, low cohesion)

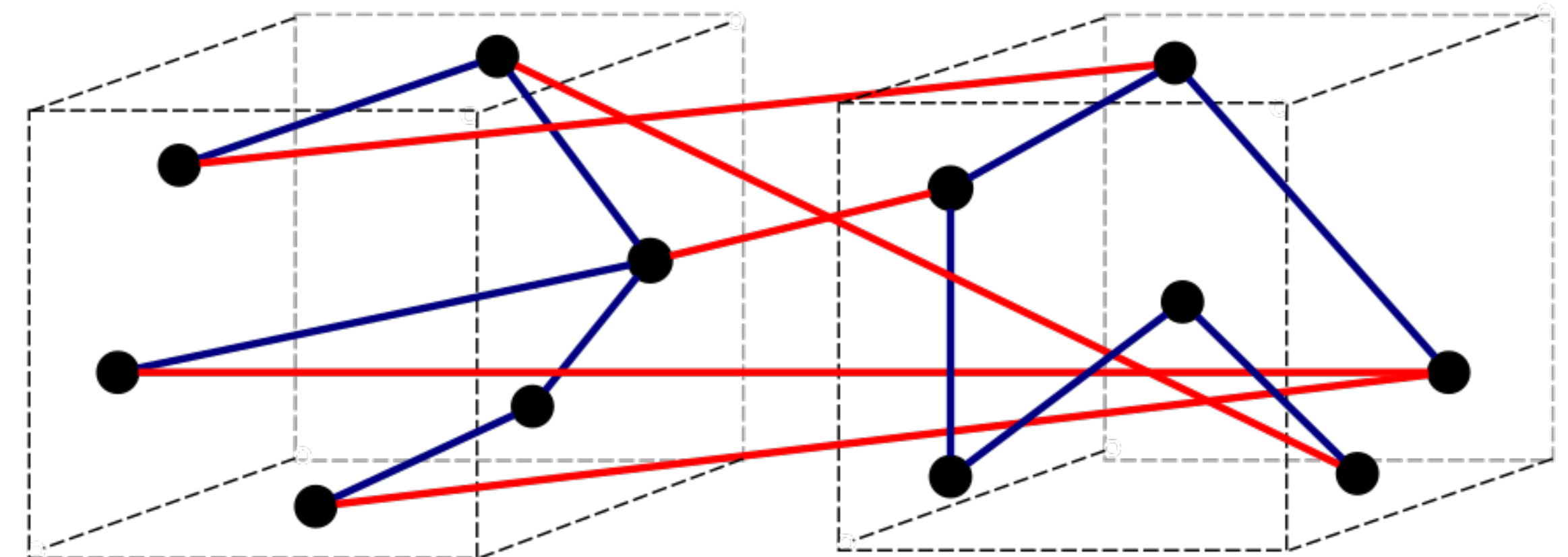
Easier to Change: Hohe Kohäsion

Textbasiertes Interface

- 💡 Kohäsion bezieht sich darauf, wie gut die Elemente innerhalb eines Moduls zusammenpassen und wie eng sie miteinander verbunden sind.
- 🔩 **Hohe Kohäsion:** die Funktionen innerhalb eines Moduls sind eng miteinander verwandt und auf ein gemeinsames Ziel ausgerichtet.
- 🔧 Ein gut kohäsives Modul sollte eine klare, abgegrenzte Verantwortung haben und eine bestimmte Aufgabe oder Funktionalität erfüllen.
- 👤 Dies erleichtert das Verständnis, die Wartung und die Wiederverwendbarkeit des Codes.

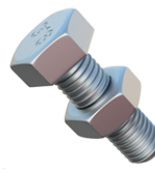


a) Good (loose coupling, high cohesion)



b) Bad (high coupling, low cohesion)

 „Genau, auch super
hilfreich für Codeopolis.“

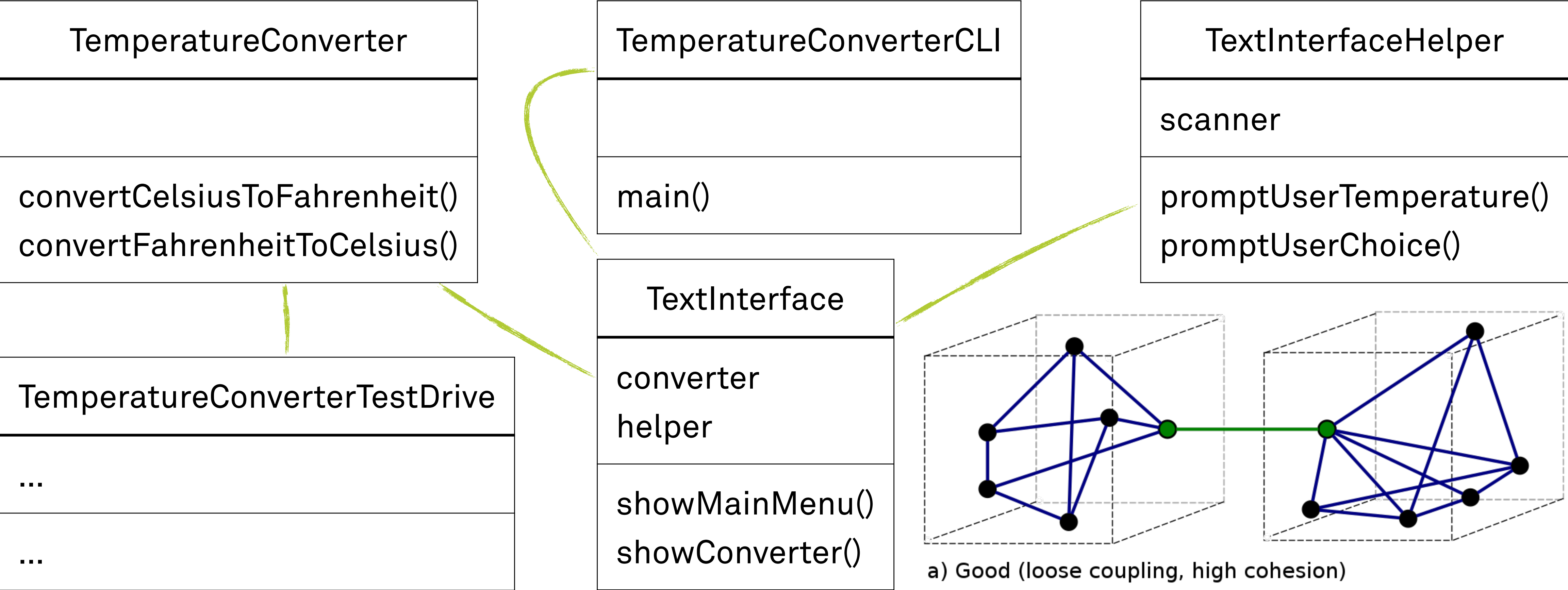
„Niedrige Kopplung  – Hohe Kohäsion  
Diese Ausprägung des Prinzips Easier to Change fördert
die Erstellung einer flexiblen, wartbaren und leicht
verständlichen Softwarelösung.“

Maxi – professionelle Entwickler:in, die andere zu Spitzenleistungen führt



Easier to Change: ↓Kopplung 🔗 und ↑Kohäsion 🛠️

💻 Textbasiertes Interface



```
1 public class TextInterface {
2
3     private TemperatureConverter converter = new TemperatureConverter();
4
5     private TextInterfaceHelper helper = new TextInterfaceHelper();
6
7     public void showMainMenu() {
8         boolean running = true;
9
10        while (running) {
11            System.out.println("==== Hauptmenü =====");
12            System.out.println("1) Umrechnung");
13            System.out.println("2) Beenden");
14
15            int choice = helper.promptUserChoice("Bitte geben Sie entweder 1 oder 2 ein");
16
17            // See: Branching with Switch Statements
18            // https://dev.java/learn/language-basics/switch-statement/
19            switch (choice) {
20                case 1:
21                    showConverter();
22                    break;
23
24                case 2:
25                    helper.quit();
26                    running = false;
27                    break;
28
29                default:
30                    System.out.println("Ungültige Eingabe!");
31                    break;
32            }
33        }
34    }
35
36    public void showConverter() {
37        System.out.println("==== Umrechner =====");
38
39        double temperature = helper.promptUserTemperature();
40
41        double fahrenheit = converter.convertCelsiusToFahrenheit(temperature);
42        double celsius = converter.convertFahrenheitToCelsius(temperature);
43
44        System.out.println(temperature + " Grad Celsius in Fahrenheit: " + fahrenheit);
45        System.out.println(temperature + " Grad Fahrenheit in Celsius: " + celsius);
46    }
47 }
48
```

STDIN

0

1

23.7

2

Output:

==== Hauptmenü =====

1) Umrechnung

2) Beenden

Bitte geben Sie entweder 1 oder 2 ein: <0>

Ungültige Eingabe!

==== Hauptmenü =====

1) Umrechnung

2) Beenden

Bitte geben Sie entweder 1 oder 2 ein: <1>

==== Umrechner =====

Bitte geben Sie die Temperatur ein: <23.7>

23.7 Grad Celsius in Fahrenheit: 74.66

23.7 Grad Fahrenheit in Celsius: -4.611111111111111

==== Hauptmenü =====

1) Umrechnung

2) Beenden

Bitte geben Sie entweder 1 oder 2 ein: <2>

› java TemperatureConverterCLI.java

==== Hauptmenü ====

1) Umrechnung

2) Beenden

Bitte geben Sie entweder 1 oder 2 ein: 0

<0>

Ungültige Eingabe!

==== Hauptmenü ====

1) Umrechnung

2) Beenden

Bitte geben Sie entweder 1 oder 2 ein: 1

<1>

==== Umrechner ====

Bitte geben Sie die Temperatur ein: 23,7

<23.7>

23.7 Grad Celsius in Fahrenheit: 74.66

23.7 Grad Fahrenheit in Celsius: -4.611111111111111

==== Hauptmenü ====

1) Umrechnung

2) Beenden

Bitte geben Sie entweder 1 oder 2 ein: 2

<2>

! Quiz

🖥 Textbasiertes Interface

- 🤔 Ich mache einige Aussagen – sind diese Aussagen richtig oder falsch?
- 👍 Bei **richtigen** Aussagen **stehen** Sie **kurz auf**!
- 👎 Bei **falschen** Aussagen **bleiben** Sie **sitzen**!



Das „Easier to Change“-Prinzip besagt, dass ein gutes Softwaredesign leichter zu ändern ist als ein schlechtes Softwaredesign.

Richtig!

Kopplung bezieht sich auf das Ausmaß, in dem einzelne Module oder Komponenten einer Software miteinander verbunden sind.

Richtig!

Eine hohe Kopplung erleichtert
Änderungen und Wartung.

Eine lose Kopplung, da dann eine Änderung in einem Modul weniger Auswirkungen auf andere Module hat.

Kohäsion bezieht sich auf das Ausmaß, in dem die Elemente innerhalb eines Moduls zusammenpassen und wie eng sie miteinander verbunden sind.

Richtig!

Ein hoch kohäsives Modul ist auf eine klar definierte Funktion ausgerichtet. Alle Elemente arbeiten zusammen, um diese Funktion zu erfüllen. Dadurch wird der Code verständlicher und leichter wartbar.

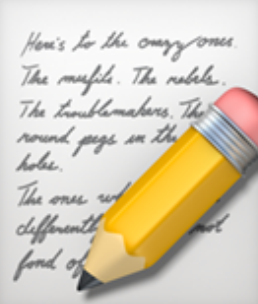
Richtig!

„Lose Kopplung und hohe Kohäsion.“ Diese Ausprägung des Prinzips „Easier to Change“ fördert die Erstellung einer flexiblen, wartbaren und leicht verständlichen Softwarelösung.

Richtig!

Eine Implementierung, die dem „Easier to Change“-Prinzip folgt, trennt beispielsweise die Anwendungslogik von der Benutzeroberfläche. Diese Trennung ermöglicht es, Änderungen an der Benutzeroberfläche vorzunehmen, ohne die zugrunde liegende Logik zu beeinflussen, und umgekehrt.

Richtig!

 Welcher **eine** Gedanke zu
„ Textbasiertes Interface“ war
für Sie am wichtigsten?
 Schreiben Sie ihn auf!

 Durchatmen und  Sortieren

 **Zielgerade**

↔ Teach Back

📖 Die Java-API kennenlernen

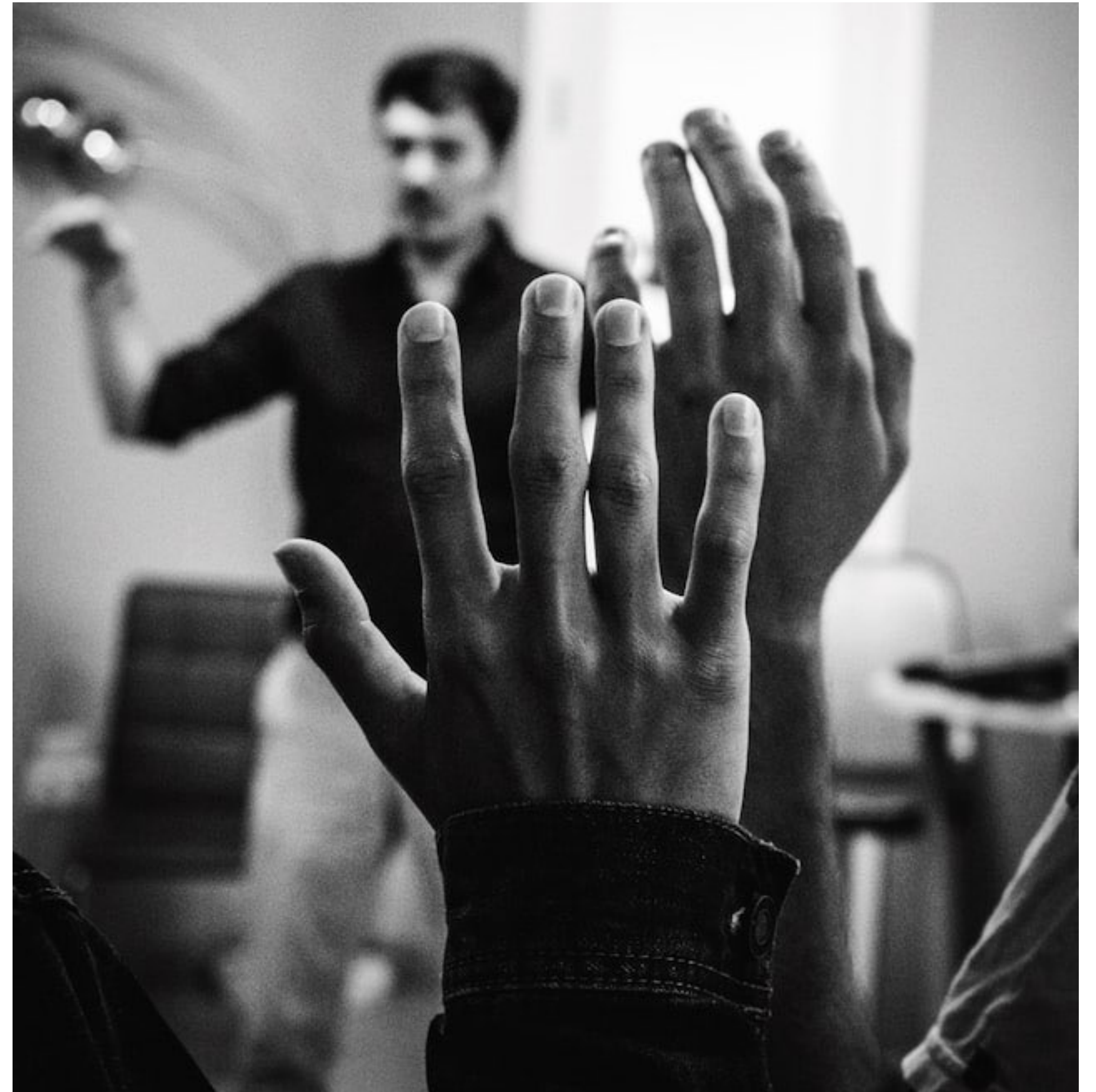
- 🎤 Beantworten Sie sich zusammen mit einer Nachbar:in gegenseitig kurz und knapp die folgenden Fragen:
- 💡 Was habe ich heute gelernt und erfahren?
- 💡 Was davon ist für mich besonders wichtig?



💡 Ihre Fragen

👉 Unsere Antworten

- Welche Fragen haben Sie zu  **Die Java-API kennenlernen?**
- Was irritiert Sie vielleicht sogar?
- Was interessiert Sie außerdem?





Lern-Logbuch



Die Java-API kennenlernen



Machen Sie sich Notizen:



Was sind für mich die wichtigsten Erkenntnisse?



Was weiß ich jetzt, was ich vorher nicht wusste?



Wie kann ich dieses Wissen anwenden?



Welche Fragen habe ich noch?



Was werde ich tun, um sie zu beantworten?



Ihr Lernerfolg – Unser Applaus

